Report Group 1: Enriching Alerts for Improved Intrusion Detection Systems

Tsvetan Kolev
TU Berlin
t.kolev@campus.tu-berlin.de

Robert Quander
TU Berlin
quander@campus.tu-berlin.de

Fatih Tekin
TU Berlin
fatih.k.tekin@campus.tu-berlin.de

***Abstract*—This report presents the tasks and solutions developed as part of the "Project Intelligent Cybersecurity Applications" course. The work was conducted by Group 1, working on Task 1, with the goal of improving intrusion detection and alert management within a cybersecurity framework.**

To provide a comprehensive overview of the project, this report details all major design decisions, from dataset selection and anomaly detection methods to alert enrichment and structured case management. Particular attention is given to the practical implementation of the final system, ensuring that both the technical rationale and operational improvements are clearly shown.

The report concludes with an outlook on future improvements and acknowledges the contributions of the authors involved in the project.

I. Introduction

In this section, we present the main goals and objectives of the project and their structured breakdown into subgoals. To ensure a clear and systematic approach, we also define the individual tasks that contribute to achieving each goal. This breakdown provides a roadmap for implementing everything while maintaining a focus on the project's main purpose: improving automatically generated alerts of an anomaly based intrusion detection system.

A. Motivation

A previous iteration of an anomaly-based intrusion detection system (IDS) was developed in an earlier class last semester. While this system demonstrated potential for detecting unknown attacks, it had a critical limitation: its alerts were too vague, offering little operational value for system administrators. The system primarily focused on detecting anomalous activity, such as port scans, without providing meaningful context or actionable insights.

The primary goal of this project is to enhance the utility of IDS alerts by making them more structured, interpretable, and actionable. Instead of merely detecting anomalies, we aim to refine the alerting process by integrating our findings with DFIR-IRIS, a system designed to enrich alerts with additional context. This includes creating structured cases with detailed information and predefined response tasks, enabling system administrators to react more effectively. By enhancing alerts with meaningful context and recommended actions, this project seeks to bridge the gap between anomaly detection and practical incident response, making IDS outputs significantly more useful for real-world security operations.

B. Description of Problem

A fundamental challenge in intrusion detection systems (IDS) is the sheer volume and lack of specificity of alerts they generate. While modern IDS solutions are highly effective at recognizing suspicious activity, their output often consists of generic, low-level security events that provide little actionable insight. Administrators are frequently bombarded with an overwhelming number of alerts, many of which lack context. As a result, distinguishing between false positives, minor anomalies, and actual security threats becomes increasingly difficult.

This problem is particularly pronounced in network scan detection, where an IDS might generate thousands of alerts for routine or benign scanning behavior. Research has shown that signature-based IDS tend to flag vast amounts of low-severity events, often failing to correlate them effectively into meaningful security incidents [6]. Without sufficient correlation and enrichment, security professionals are left to manually analyze and interpret raw alerts, leading to alert fatigue — a condition where administrators become desensitized to notifications due to their sheer volume. This fatigue increases the likelihood of critical security incidents being missed [7] [14].

Efforts have been made to mitigate this issue through alert correlation techniques. Multi-stage intrusion correlation models attempt to establish logical connections between individual alerts, helping to identify broader attack patterns rather than treating every event in isolation [9]. Similarly, statistical-based and hybrid-based correlation methods provide mechanisms for filtering and prioritizing alerts, aiming to reduce the number of irrelevant notifications security teams must process [15]. However, even with these improvements, a significant portion of IDS alerts remains too unspecific to be operationally useful.

This lack of specificity not only leads to reduced situational awareness but also has direct consequences on human performance. Security analysts are expected to monitor and respond to alerts continuously, yet when the vast majority of these alerts lack relevance, they struggle to maintain focus. Over time, this leads to cognitive overload and decision fatigue, increasing the likelihood of human error. Studies have highlighted that alert volume and quality remain major obstacles in IDS effectiveness, necessitating ongoing refinement of how alerts are processed and presented to operators [6].

Addressing this problem requires not just improved detec-

tion but also better structuring and prioritization of alerts. Instead of simply flagging individual network events, an IDS must contextualize its findings, ensuring that only meaningful, actionable alerts reach administrators. Without such improvements, IDS solutions risk becoming counterproductive—flooding security teams with excessive notifications while failing to provide the clarity needed to respond effectively to real threats.

C. Goals

1) Subgoal 1: To systematically accomplish the main goal, the project was broken down into several predefined subgoals, each contributing to the overall functionality of the system:

The first subgoal was to select an intrusion detection dataset that contained port scanning activity and could be replicated reliably. The ability to regenerate data in a consistent format was critical for testing and evaluation. This requirement ensured that experimental results remained reproducible across different setups, reducing the risk of inconsistencies in detection performance.

2) Subgoal 2: The second subgoal was to develop an anomaly detection system specifically designed to identify port scans. Since anomaly detection can be prone to false positives and false negatives, it was important to clarify that optimizing detection accuracy was not the project's primary focus. Instead, the goal was to ensure that the system could detect at least some port scanning activity reliably, providing a foundation for further enrichment of the generated alerts.

3) Subgoal 3: The third subgoal involved creating a controlled test environment to validate the detection system. This required setting up a small network of virtual machines (VMs) on which port scans could be conducted under controlled conditions. By running scans against these VMs and processing the resulting network traffic, we could assess the performance of our IDS in a structured and repeatable manner. To provide an additional reference point, Suricata was also installed in the environment, allowing for a comparative analysis of rule-based and anomaly-based detection methods. The detection results from both systems were stored in the EVE JSON format, ensuring compatibility with subsequent processing steps. Furthermore, we aimed to explore techniques to enrich the generated alerts with meaningful additional information, making them more informative for security analysts.

4) Subgoal 4: The fourth subgoal was to integrate DFIR-IRIS into the workflow to manage and categorize alerts. The objective was to configure the system to automatically create structured cases from detected alerts and investigate whether DFIR-IRIS provided additional mechanisms for enhancing alert information beyond what was possible in EVE JSON. This integration aimed to bridge the gap between raw detections and structured security incident management, ensuring that alerts could be used effectively in real-world forensic analysis and response.

These subgoals, though distinct, were interdependent, forming a structured approach to improving the IDS's effectiveness and operational utility.

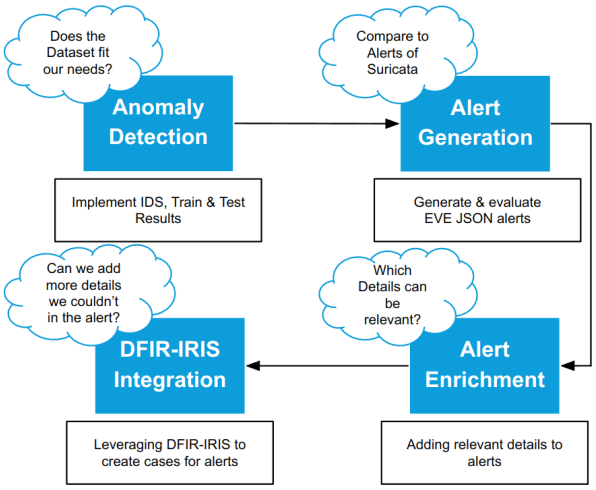

Fig. 1. The detection pipeline illustrates the structured flow of data from initial anomaly detection to alert enrichment and integration with DFIR-IRIS for security incident management.

D. The Detection Pipeline

To further clarify the breakdown of tasks and their implementation, we defined a detection pipeline that structures the flow of data from initial detection to final alert processing. The pipeline is visualized in Figure 1.

This pipeline ensures that the detection system follows a structured and reproducible workflow, with each stage contributing to the final enriched alerts.

II. The Dataset

One of the earliest and most important tasks to tackle was to find a fitting dataset that would help us design and evaluate our intrusion detection system. The dataset needed to contain port scanning activity, be structured in a format that allowed for effective processing, and ideally be replicable, ensuring that our experiments could be reproduced.

Initially, we chose the MAWI dataset [1] as our primary data source. The dataset appeared highly promising due to several factors. It contained a large volume of network traffic, providing extensive data for anomaly detection. Not only did it contain a rather big number of network traffic, but more importantly also very recent traffic, such as from 2020 [4]. Given that there is also a big number of older traffic, one could even observe historical trends, such as how traffic changed during the COVID period [2]. Also, this recent network traffic data would ensure that our IDS would work with more modern types of scans. Additionally, the dataset was labeled, allowing us to directly compare detected anomalies with known attack patterns. Another major advantage was that the accompanying research paper provided a clear and well-structured explanation of the dataset, making it easier to understand and apply.

However, despite these advantages, we encountered several challenges that ultimately led us to reconsider our choice. The first and most immediate problem was that the dataset's downloads were extremely large and slow, making it nearly

impossible to efficiently even obtain the data. Even after overcoming these logistical issues, we found that training on MAWI data was not as successful as we hoped it would be, as the structure and content of the dataset did not seem to align well with our approach. Most importantly however, the portscan density was extremely low, giving us very little data to work with. Given these limitations, continuing with MAWI would have introduced significant obstacles to the experimental process.

Due to these issues, we opted to use the UNSW-NB15 (also referred to as UNSW15 henceforth) dataset presented by UNSW Sydney in [5] instead. The dataset was first introduced by Moustafa and Slay, who designed it to reflect realistic network conditions and address the limitations of previous datasets like KDD99. While not a perfect solution, UNSW15 provided a more practical balance between usability and relevance. The dataset was still recent enough to be applicable to modern network security challenges. More importantly, while very complex, it was at least possible to replicate, which we will get into in chapter III-A, ensuring that we could generate consistent results across different runs. Although UNSW15, like MAWI, contained a substantial amount of data. The number of portscans was still very little, which was not useful to our task. Nontheless it was more manageable, allowing us to filter and preprocess the relevant portions more effectively. Additionally, training our IDS on UNSW15 yielded better results, making it the more viable choice for our system's development and evaluation.

By selecting UNSW15, we ensured that our dataset supported effective model training, addressing the key limitations that had made MAWI impractical for our approach.

III. Implementation

This section gives a brief overview of the systems used to accomplish the extended IDS. Both the components and the ideas used will be taken into account here in order to lay the foundation for the entire functionality.

A. The Docker Environment

To ensure a reproducible and stable foundation for System, we containerized everything from our system up using Docker. This approach encapsulates all necessary dependencies, ensuring consistent execution across different systems and simplifying the setup process. Additionally, it allows for easy deployment and testing without the risk of dependency conflicts or system-specific configurations.

Inside the Docker container, we have multiple scripts, each serving a specific purpose. The first script is responsible for parsing the PCAP file, which the user must load into the filesystem. To efficiently handle PCAP processing, our system leverages network monitoring tools such as Argus, Zeek, and PyShark. These tools extract relevant network features, which are then used for anomaly detection.

During testing, we observed that not all PCAP files were parsed successfully, though we couldn't fully determine the cause, possibly related to file corruption or format inconsistencies. To address this, we implemented exception handling to skip problematic PCAP files instead of halting the entire process. Once the data has been processed, the script generates a CSV file, which is then used by our trained machine learning model to predict whether the network traffic contains malicious activity.

Initially, we planned to train the model directly within the Docker container. However, we realized that retraining the model every time we wanted to generate alerts was impractical. To improve efficiency, we split the original script, `final_alerts_script.py`, into two separate scripts: `train_model.py` for training and `generate_alerts.py` for prediction and alert generation.

The model is trained once and saved persistently. Each time we need to generate alerts, we simply load the pre-trained model and apply it to the processed network traffic data. This approach significantly reduces computational overhead while maintaining the system's responsiveness.

The `generate_alerts.py` script processes the CSV file and detects suspicious network activity, particularly port scans. The script outputs alerts in JSON format, which are stored in the designated `CaseManagement/alerts` directory. This structured output facilitates further analysis and serves as an input for DFIR-IRIS. Additional details on how we utilise DFIR-IRIS are provided in III-E. In the `CaseManagement` directory are the necessary scripts for setting up and uploading a new case per API to the DFIR-IRIS.

In summary, our Docker environment forms the backbone of the system, ensuring a consistent and reproducible platform for both data processing, machine learning tasks and case management. By containerizing all dependencies and segregating functionalities into dedicated scripts, we not only mitigate deployment challenges but also enhance system maintainability and scalability. Furthermore, this streamlined setup provides a flexible foundation for integrating future enhancements and adapting to evolving operational requirements.

B. PCAP-Based Port Scan Detection

At first, we developed the system in a static manner, where the IDS processed a predefined dataset and applied the method described in Chapter III-C to distinguish between malicious and benign traffic. This allowed for a structured evaluation of detection performance but did not align with the broader goal of handling real-world network activity more effectively. While a fully dynamic solution, where live traffic is continuously monitored and processed in real-time, was considered, we ultimately opted for a more controlled approach using packet captures (PCAPs) instead.

The decision to use PCAPs rather than direct live traffic monitoring was due to several factors. Firstly, relying on recorded network traffic allowed for better reproducibility, making sure that different tests could be undertaken under identical conditions. This consistency was critical for evaluating detection performance and debugging potential issues

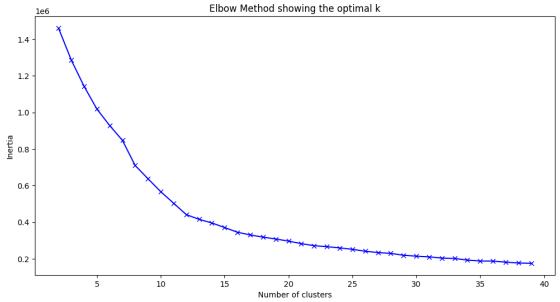

Fig. 2. Using the elbow-method, we came to an optimum of 13

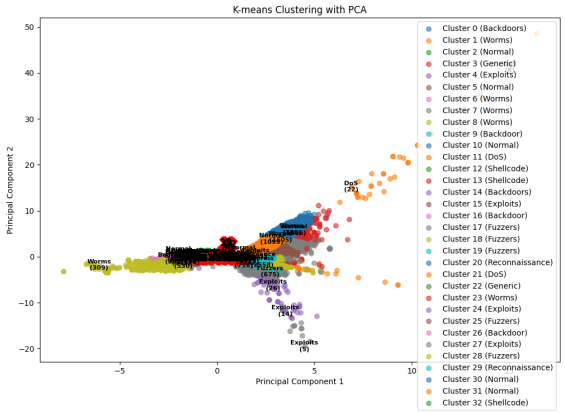

Fig. 3. Clustered Alert Groupings With 34 Clusters After K-Means Clustering

without the variability introduced by live network conditions. Additionally, working with PCAPs reduced the complexity of our implementation, as real-time packet capture and streaming processing would have introduced significant overhead in terms of infrastructure and resource management.

To integrate PCAPs into our workflow, we structured the detection pipeline to ingest recorded network traffic, process it using our anomaly detection system, and generate alerts based on observed patterns. This method allowed us to simulate real-world conditions while maintaining control over input data, ensuring that the IDS could effectively analyze different types of scanning behavior. The ability to replay captured traffic also enabled us to refine detection strategies and validate results without relying on an unpredictable live network.

One of the key benefits of this approach was that it still allowed for long-term correlation of network events, even if it was not strictly real-time. By processing large volumes of stored traffic, we could observe scanning behavior over extended periods, identifying slow or distributed scan attempts that might otherwise be difficult to detect. Additionally, the structured format of PCAP files ensured compatibility with other security tools, enabling cross-validation of our IDS results with rule-based detection systems such as Suricata.

C. The classifiction of the collected data: k-means

To detect port scans, we employed k-means clustering. Since the data we want to classify later is not as structured as the training data might be, we decided to go with this unsupervised approach. In opposition to a more traditional rule-based detection method, which can be inflexible and require constant updates to keep up with the evolution of new scanning techniques, we wanted to implement a system able to detect anomalies in network traffic without relying on those predefined rules.

K-means has several shortcomings, particularly in result precision. We did detect a high false positive as well as false negative rate. A more detailed discussion about the concrete problems and the fixes we applied, can be found in chapter IV-B.

Given the rather unsatisfactory detection rates produced by k-means in its initial configuration, we explored alternative models, including Isolation Forests and Support Vector Machines (SVMs). However, both approaches yielded even worse detection performance. Isolation Forests, while effective in detecting global outliers, struggled with the subtle variations inherent in port scanning behavior. SVMs, on the other hand, required a well-defined decision boundary that was difficult to establish given the highly imbalanced nature of our dataset. These limitations led us to return to k-means clustering since it was the most viable approach despite its shortcomings.

At this stage, we revisited our clustering configuration, particularly the optimal number of clusters. Initially, we applied the elbow method, as seen in figure 2, which suggested an optimal cluster count of around 13. To further assess clustering performance, we also considered the silhouette score, which evaluates the cohesion and separation of clusters. As a third option, we discussed limiting ourselves to only the feature dimensions that were presented by Yin et al. in 2022 [3], but after not seeing a huge improvement for our use-case and being undecided if the features left out might in some situation present some useful data, we ultimately decided against using it.

However, all three of these methods provided only heuristic insights and did not directly align with our goal of isolating port scan activity, specifically the ”Reconnaissance” attack category in the UNSW15 dataset. Through experimentation, we determined that more than 25 clusters, as seen in figure 3 were needed for meaningful separation. This deviation from the initially recommended values ensured that the clustering model captured the required level of granularity to distinguish scanning behavior from normal traffic.

By refining the clustering strategy and minimizing feature noise, we ultimately improved the performance of k-means sufficiently for use in the IDS. While k-means was not perfect, it provided the best trade-off between flexibility and detection capability within the constraints of our project.

D. Suricata and EVE JSON

Although our primary goal was to develop an IDS system using machine learning techniques, we also set up Suricata as

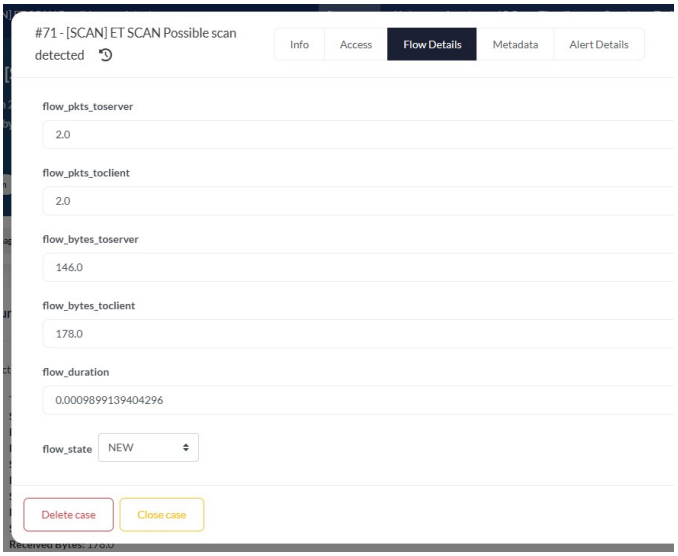

Fig. 4. Custom Attribute Page for Case Structuring in IRIS (Focus: Flow Details)

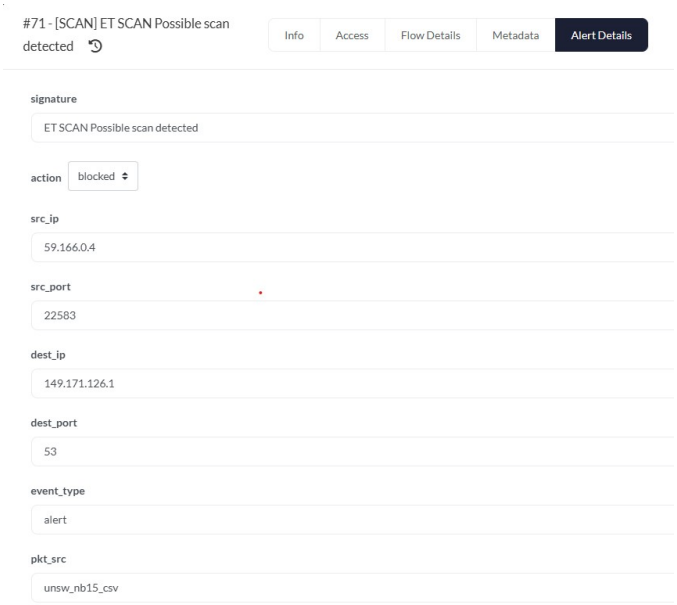

Fig. 5. Custom Attribute Page for Case Structuring in IRIS (Focus: Alert Details)

a secondary detection mechanism. This decision was made by taking two main points into consideration: first, to compare the alerts generated by our k-means-based approach with those produced by a widely used rule-based IDS, and second, to enhance the meaningfulness of the alerts we ultimately forwarded to DFIR-IRIS.

Suricata was deployed within a Docker environment. To still allow structured logging and compatibility with our later processing pipeline, we configured Suricata to output its alerts in EVE JSON. This format allowed us to parse and compare everything more efficiently and also to make sure that the integration with the rest of our system worked.

By running both our machine learning-based approach and Suricata, we could evaluate the differences in detection performance. While Suricata uses predefined rules to flag suspicious behavior, our unsupervised k-means clustering approach attempted to detect anomalies without such constraints. This comparative analysis helped us better understand the strengths and weaknesses of both methods. Also, it allowed us to refine our own alerting system, ensuring that the alerts we generated were not only statistically justified but also actionable in a real-world security context.

Suricata was seen as more of a benchmarking tool giving us something that could be achieved without the enrichment we wanted to bring to the table.

E. DFIR-IRIS: Ordering the alerts

We integrated DFIR-IRIS into our environment to organize and process alerts from our IDS system, as required for our project. This integration provided a structured approach to manage and analyze the generated alerts as seen in figures 4 and 5.

The biggest challenge was to connect our generated alerts with the DFIR-IRIS system in a way that allowed efficient processing without introducing unnecessary complexity into our detection pipeline.

Since the entire environment is containerized using Docker, DFIR-IRIS was deployed as a separate container within our setup. This allowed it to interact easily with our other services that were already running while maintaining modularity and scalability. Communication between our IDS and DFIR-IRIS was established via an API, ensuring that alerts could be forwarded and processed in real-time. The IDS system, upon detecting an event classified as a potential port scan, would format the alert according to the required DFIR-IRIS schema and send it to the respective endpoint.

A crucial design decision was to structure our incident handling around cases rather than individual alerts. We opted for a one-alert-per-case model. This ensured that each alert was treated with full context, allowing for precise tracking and response. Relevant info is found in Flow Details, as seen in 4.

To enhance the efficiency of incident response, we used custom attributes to enrich cases with relevant metadata. Additionally, we implemented case templates, which dynamically assigned predefined tags and tasks based on the severity of the alert. This templating mechanism streamlined the workflow by ensuring that each case had a structured response plan tailored to its severity level. This approach reduced manual overhead while maintaining consistency in incident handling.

F. Enrichment of the alerts

A raw alert often does not provide the necessary context to be immediately acted upon. To improve the quality of the alerts generated by our IDS, we implemented an enrichment process before forwarding them to DFIR-IRIS. This step was crucial in ensuring that alerts contained enough meaningful

information for effective analysis and response. Instead of simply passing on detections, our system augmented them with additional metadata like severity, contextual information and making it easier to distinguish between benign activity and actual threats.

The enrichment process was deployed within our Docker environment, operating as an intermediary between the detection system and DFIR-IRIS. This modular design allowed us to keep alert processing flexible and scalable while avoiding unnecessary dependencies between detection and response components.

IV. Evaluation

In this section we give an evaluation of the work we did, analyzing the results we found during the completion of this project, taking a look at the problems we faced before drawing a conclusion and providing an outlook for future work in the field as well as ideas of what other projects could be created on basis of this one.

A. Results

This project contributes to the field of network intrusion detection by refining the application of unsupervised learning for port scan detection and improving the practical usability of IDS alerts. While developing our anomaly-based detection system, we employed k-means clustering with the primary objective of ensuring reliable alert generation. Enhancing the algorithm's performance and results was not our main focus. Instead, our work improves the operational effectiveness of IDS alerts by incorporating alert enrichment and structured case handling. Many existing IDS implementations generate isolated, low-level alerts that overwhelm administrators without providing clear guidance for response. By integrating DFIR-IRIS into the alert management process, we enhanced alert enrichment and structured case handling. Many existing IDS implementations generate isolated, low-level alerts that overwhelm administrators without providing clear guidance for response. To address this, we introduced custom case templates, ensuring that cases are structured and separated based on their severity. This categorization allows system administrators to handle incidents with a tailored response plan, as different severity levels automatically generate predefined tasks and tags specific to the nature of the alert. By bridging the gap between raw anomaly detection and real-world security operations, these improvements make IDS solutions more actionable for security teams, reducing alert fatigue and improving the overall effectiveness of intrusion monitoring.

B. Problems and problem-analysis

During the implementation of this system, we encountered numerous problems in several areas that needed solving.

One of the biggest issues was the low detection performance of our IDS using k-means clustering. Initially, the model produced a high rate of both false positives and false negatives, making it rather unreliable. This problem was largely due to the nature of the dataset, as k-means relies on clearly separable clusters, which were not always present in our network traffic data. To address this, we undertook a process of fine-tuning the model, focusing on adjusting hyperparameters such as the number of clusters and the initialization method. Additionally, we reduced the number of features used in clustering by selecting only the most relevant columns, trying to remove noise from the dataset. While these adjustments did not completely resolve the issue, they led to noticeable improvements in detection rates, making k-means a more viable solution for our specific task.

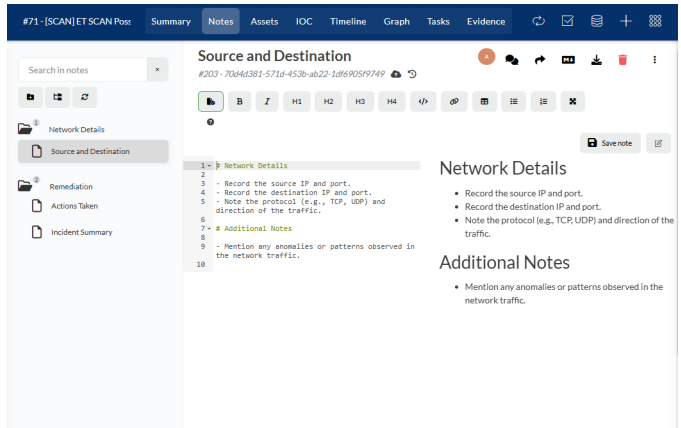

Fig. 6. Notes Section with Incident Response Guidance in IRIS

Additionally, we encountered difficulties in replicating the UNSW15 dataset, which complicated both the training and evaluation of our IDS. To address these challenges, we used Argus and Zeek, which helped resolve the replication issues. However, installing and configuring these tools was not straightforward. To ensure consistency and make the setup more accessible for everyone, we containerized the entire process using Docker. This approach not only mitigated installation difficulties but also ensured that data could be generated in a consistent manner across different environments. By doing so, we minimized issues related to varying configurations or preprocessing steps, improving reproducibility and enhancing the performance of our IDS, as the data was cleaned and formatted in a more structured manner before being fed into our clustering algorithm.

We also faced a challenge with the insufficiency of the standard IRIS case template and its predefined attributes. The default structure did not provide the level of detail and categorization we needed to effectively organize and respond to alerts. To address this, we created our own custom case templates and defined custom attributes that allowed us to better structure alerts and provide more meaningful context. Additionally, we used the Notes section of each case to document relevant details and provide guidance on appropriate response actions, as seen in figure 6.

V. Related Work

The detection and management of network intrusions have been extensively studied in cybersecurity research. Various approaches have been proposed to enhance the accuracy and efficiency of Intrusion Detection Systems, also for port scan detection. Many studies have a particular focus on the application of unsupervised learning techniques, including k-means clustering, to identify malicious activities, while others explore alternative machine learning methods.

One of the most widely referenced datasets for intrusion detection research is UNSW-NB15, which we also used for our objective. It provides a modern and comprehensive set of network traffic samples, including normal activity and various attack types. Studies have explored different feature selection techniques to improve the effectiveness of machine learning models applied to UNSW-NB15, such as deep learning-based feature selection [17] and hybrid selection methods combining statistical and machine learning approaches [3].

A. K-Means Clustering for Intrusion Detection

Unsupervised learning techniques such as k-means clustering have been frequently applied in IDS research to detect anomalies in network traffic. Münz et al. [11] proposed one of the earliest approaches using k-means to identify attack patterns in flow records. More recently, Karim et al. [12] demonstrated how k-means can be leveraged for IDS anomaly detection, emphasizing the challenges posed by noisy data and feature selection. Mutanov et al. [16] extended this work by incorporating sequential minimal optimization techniques to refine clustering performance and reduce false positives.

However, one of the major limitations of k-means is its sensitivity to feature selection and cluster count. Research by Fawaz et al. [13] focused on detecting slow port scanning using k-means-based anomaly detection, showing how clustering techniques struggle with subtle attack patterns. This aligns with our findings that adjusting the number of clusters is crucial for isolating specific attacks such as reconnaissance scans.

B. Alternative IDS Machine Learning Models

While k-means clustering is a widely used approach, other studies have explored alternative machine learning models for IDS applications. Shon et al. [14] proposed a semi-supervised alert filtering method, which helps reduce false positives in high-volume IDS environments.

However, studies have shown that unsupervised methods remain more flexible in detecting novel attack patterns. Bakar and Kijsirikul [15] introduced an advanced port scanning technique for improving network security, emphasizing that traditional rule-based IDS approaches are often ineffective against modern, stealthy scanning methods. Their findings suggest that hybrid models combining clustering and supervised learning techniques may offer a better balance between accuracy and generalizability.

C. Alert Correlation and Processing in IDS

One of the key issues in intrusion detection is the large volume of generated alerts, leading to alert fatigue among security analysts. Viinikka et al. [7] proposed using time-series modeling to aggregate and process IDS alerts more efficiently, reducing the workload on analysts. Similarly, Albasheer et al. [6] reviewed various alert correlation techniques, including multi-stage correlation models and statistical-based approaches, which improve threat detection by associating individual alerts with larger attack patterns.

Another difficulty in alert management is detecting and prioritizing relevant scans within large-scale network traffic. Feng et al. [9] proposed a behavior-based method to detect distributed scan attacks in darknets, which monitors scanning activity across multiple destination ports. Meanwhile, Bakar and Kijsirikul [15] explored programmable hardware-based scanning techniques, improving IDS efficiency while reducing computational overhead.

VI. Conclusion

A. Summary

This project set out to enhance the operational utility of an IDS, with a particular focus on detecting and managing port scan alerts. While anomaly-based IDSs have shown some potential in identifying previously unknown attacks, they often suffer from poor alert specificity, overwhelming administrators with vague and too many notifications. Our approach aimed to address this issue by refining the detection process, improving alert enrichment, and integrating structured case management.

To achieve this, we first selected an appropriate dataset for training and evaluation. Initially, we considered the MAWI dataset, which offered large volumes of labeled network traffic, but we ultimately transitioned to UNSW-NB15 due to download limitations, irreproducibility, and low detection performance in our experiments. UNSW-NB15, while not without its own challenges, provided a structured and replicable environment, allowing us to systematically test and improve our IDS.

For anomaly detection, we employed k-means clustering, leveraging its unsupervised nature to identify scanning behavior without relying on predefined signatures. However, k-means introduced high false positive and false negative rates, requiring fine-tuning and feature selection to improve results. Through iterative testing, including trials with SVMs and Isolation Forests, we refined our clustering strategy, ultimately determining that a higher cluster count than initially recommended by heuristic methods was necessary to effectively capture reconnaissance activity.

Beyond detection, we integrated Suricata to compare rule-based and anomaly-based alerts, generating structured event logs in EVE JSON format. This allowed us to benchmark our IDS performance and identify areas for improvement. To further enhance the utility of alerts, we introduced our enrichment process. This step aimed to provide better contextual information for security analysts.

A major challenge was ensuring a stable and reproducible deployment environment. We encountered difficulties in maintaining a reliable DFIR-IRIS instance, which we resolved by fully containerizing the system in Docker. This not only streamlined deployment but also facilitated consistent alert case management. Additionally, we faced challenges in dataset replication and alert structuring, which were addressed through targeted preprocessing and custom case templates in DFIR-IRIS, respectively.

Through this project, we demonstrated that alert correlation, contextual enrichment, and structured case management significantly enhance the operational value of an IDS. While k-means clustering alone proved insufficient for high-accuracy detection, its integration into a broader security framework, alongside rule-based detections and enrichment techniques, resulted in a more practical and useful alerting system.

B. Future Work

While this project successfully improved port scan detection and alert management, certain areas require further refinement. The k-means clustering approach, despite improvements, still produced false positives and false negatives, suggesting that hybrid models combining supervised and unsupervised learning could enhance detection accuracy. Additionally, real-time processing capabilities could be expanded to allow for continuous monitoring without introducing excessive overhead.

A key area for improvement is case management. Currently, alerts are enriched, but additional automation could improve how attacks are categorized based on the targeted port and protocol. This would help differentiate between low-risk and high-risk scans, allowing for a prioritized response. Furthermore, integrating custom case templates that generate specific tasks based on attack severity would streamline incident handling. Ensuring that alerts are properly linked to cases would also provide a clearer picture of ongoing security events, making it easier to track attack progression.

Finally, improving alert correlation mechanisms remains an important next step. While comparing rule-based detections (Suricata) with anomaly-based clustering provided insights, a more structured correlation process could further reduce alert fatigue and present meaningful attack patterns.

By addressing these aspects, the IDS framework can be further refined for greater accuracy, usability, and scalability, ensuring it is both practical and effective in real-world cybersecurity applications.

References

[1] R. Fontugne, P. Borgnat, P. Abry, and K. Fukuda, "MAWILab: Combining Diverse Anomaly Detectors for Automated Anomaly Labeling and Performance Benchmarking," in *Proc. 6th Int. Conf. Emerging Netw. Experiments Technol. (CoNEXT)*, Philadelphia, PA, USA, 2010, pp. 1–12. doi: 10.1145/1921168.1921179.

[2] L. Schumann, T. Doan, T. Shreedhar, R. K. P. Mok, and V. Bajpai, "Impact of Evolving Protocols and COVID-19 on Internet Traffic Shares," *ArXiv*, vol. abs/2201.00142, 2022. [Online]. Available: https://arxiv.org/abs/2201.00142

[3] Y. Yin, J.-S. Jang, W. Xu, A. Singh, J. Zhu, F. Sabrina, and J. Kwak, "IGRF-RFE: A hybrid feature selection method for MLP-based network intrusion detection on UNSW-NB15 dataset," *Journal of Big Data*, vol. 10, pp. 1–26, 2022. doi: 10.1186/s40537-023-00694-8.

[4] MAWI Samplepoint-G, "MAWI Samplepoint-G Traffic Data - March 18, 2020," Available: https://mawi.wide.ad.jp/mawi/samplepoint-G/2020/202003181400.html. [Accessed: Jan 4, 2025].

[5] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *2015 Military Communications and Information Systems Conference (MilCIS)*, 2015, pp. 1–6. doi: 10.1109/MilCIS.2015.7348942.

[6] H. Albasheer, M. M. Siraj, A. Mubarakali, O. E. Tayfour, S. Salih, M. Hamdan, S. Khan, A. Zainal, and S. Kamarudeen, "Cyber-Attack Prediction Based on Network Intrusion Detection Systems for Alert Correlation Techniques: A Survey," *Sensors (Basel, Switzerland)*, vol. 22, 2022. doi: 10.3390/s22041494.

[7] J. Viinikka, H. Debar, L. Mé, A. Lehikoinen, and M. Tarvainen, "Processing intrusion detection alert aggregates with time series modeling," *Inf. Fusion*, vol. 10, pp. 312–324, 2009. doi: 10.1016/j.inffus.2009.01.003.

[8] H. G. Shon, Y. Lee, and M. Yoon, "Semi-Supervised Alert Filtering for Network Security," *Electronics*, 2023. doi: 10.3390/electronics12234755.

[9] Y. Feng, Y. Hori, K. Sakurai, and J. Takeuchi, "A behavior-based method for detecting distributed scan attacks in darknets," *Inf. Media Technol.*, vol. 8, pp. 843–854, 2013. doi: 10.2197/ipsjjip.21.527.

[10] R. A. Bakar and B. Kijsirikul, "Enhancing Network Visibility and Security with Advanced Port Scanning Techniques," *Sensors (Basel, Switzerland)*, vol. 23, 2023. doi: 10.3390/s23177541.

[11] G. Münz, S. Li, and G. Carle, "Traffic anomaly detection using k-means clustering," in *GI/ITG Workshop on IP Operations and Management (IPOM)*, 2007.

[12] S. Karim et al., "Implementation of K-Means Clustering for Intrusion Detection," *ResearchGate*, 2019.

[13] K. Fawaz et al., "Slow Port Scanning Detection," *IAS 2011*.

[14] H. G. Shon, Y. Lee, and M. Yoon, "Semi-Supervised Alert Filtering for Network Security," *Electronics*, 2023. doi: 10.3390/electronics12234755.

[15] R. A. Bakar and B. Kijsirikul, "Enhancing Network Visibility and Security with Advanced Port Scanning Techniques," *Sensors (Basel, Switzerland)*, vol. 23, 2023. doi: 10.3390/s23177541.

[16] V. Karyukin, G. Mutanov, Z. Mamykova, and G. Nassimova, "On the development of an information system for monitoring user opinion and its role for the public," *Journal of Big Data*, vol. 9, no. 1, Nov. 2022. doi: 10.1186/s40537-022-00660-w.

[17] K. Venkatachalam and P. Jacob, "UNSW-NB15 dataset feature selection and network intrusion detection using deep learning," *ResearchGate*, Jan. 2019.

Appendix

Link to GitLab

This is the link to the branch that should be used for review and testing. GitLab Repository

Instructions on dependencies, running and use of the project

In the branch, you can see multiple README files containing detailed instructions on how to install dependencies required for running the code and detailed instructions on how to use the project results. GitLab Repository

Workload

The workload for the work on the tasks was relatively balanced. Between all the programming and architecture tasks, the writing of the report and the presentations as well as the planning, the group working always took great care to include everybody and distribute tasks as well as possible.

The writing of the report was a group effort as well, and the content of each section was discussed and carefully planned in advance. Since it is given as a requirement to specify who authored which part, we provide a table below.

	I-A	I-B	I-C	I-D	II	III-A
Tsvetan	x		x			x
Robert		x		x	x	
Fatih	x			x	x	
	III-B	III-C	III-D	III-E	III-F	IV-A
Tsvetan			x	x		x
Robert	x			x		x
Fatih	x	x			x	
	IV-B	V-A	V-B	V-C	VI-A	VI-B
Tsvetan	x		x		x	
Robert		x		x	x	
Fatih	x			x		x

In order to ensure a coherent and cohesive output without breaks in style or form, we decided to carefully employ ChatGPT in order to improve upon the language. Here, we did not grant ChatGPT any authorship over actual content of the report, but instead checked how specific parts might be improved without changing the message or content.